

Atol implementacija u Python 3

Faza 3

Link za projekat: <https://github.com/ikugo-dev/vi-projekat>

U okviru prve faze smo implementirali sve funkcije koje su neophodne za predstavljanje stanja table, unos početnih parametara igre, i unos poteza.

U okviru druge faze smo obezbedili proveru kraja igre.

U okviru treće faze smo implemetirali računanje heruristike i minmax algoritam sa alfa beta odsecanje kako bi smo pronašli najbolji potez AI protivnika.

Nismo koristili eksterne biblioteke za implementaciju

Sekvenca događaja.....	2
Glavna ("__main__") funkcija:.....	2
Kreiranje početnog stanja.....	2
Potez računara.....	3
Računanje heuristike.....	3
Nepovezana heuristika.....	3
Povezana heuristika.....	3
Prikazivanje table.....	4
Provera kraja.....	4
Struktura podataka:.....	5
Stanje igre.....	5
Tabla.....	5

Marko Petrović - 19317

Aleksandar Ristić - 19340

Nina Paunović - 19515

Sekvenca događaja

Glavna (“__main__”) funkcija:

1. Traži se jedna od validnih veličina tabli od strane korisnika
2. Kreira se početno stanje igre na osnovu zadate veličine
3. Pita se korisnik da li je protivnik drugo ljudsko biće ili kompjuter
 - a. Ako je izabran kompjuter, pita se korisnik koju “težinu” (dubina / kompleksnost rada VI-je) želi
4. Pita se korisnik da li želi da igra prvi
5. Ulazi se u glavni game loop (za sada je kraj nakon 999 poteza zbog toga što ne postoji provera da li je igra dostigla kraj)
6. Uzima se potez od adekvatnog igrača:
 - a. Od strane korisnika se uzima ulaz sa tastature
 - b. Od strane kompjutera se poziva minmax funkcija
7. Menja se stanje igre na osnovu poteza
8. Proverava se da li trenutni igrač ima most koji spaja dva nesusedna ostrva, i (ako postoji most) ispisuje to na ekranu i izlazi iz igre

Kreiranje početnog stanja

Pozivanjem ***create_starting_state(player_one_first: bool, size: int) -> GameState***, mi kreiramo objekat koji predstavlja stanje naše igre. Unutar njega se poziva ***create_starting_graph(size: int) -> dict[Point, Node]*** funkcija koja predstavlja korake za kreiranje grafa.

1. ***build_grid(size: int)*** funkcija - generiše 2D matricu koja sadrži sva validna polja za igru.
2. ***generate_points(size: int, grid: dict[str, list[int]])*** funkcija - uzima tu 2D matricu i pretvaraju u listu tačaka koji su zapisani kao i na prezentaciji to jest jedno slovo jedan broj a ne dva broja kao u 2D listi
3. ***build_graph(points: list[Point])*** - Za svaku tačku u Listi tačaka inicijalizuje čvor koji treba tu da se nalazi i spaja ih sa svim susednim čvorovima

Potez računara

Ako je vreme na računar da napravi potez, poziva se **`get_computer_move(state: GameState, depth=3, verbose=True)`** -> **`Point`** funkcija koja na bezbedan način poziva **`minmax_alpha_beta(state: GameState, depth: int, alpha: int = -INF, beta: int = INF, is_maximizing: bool = True)`** -> **`tuple[int, GameState | None]`**.

Alfa beta odsecanje radi tako što se rekurzivno poziva do određene dubine ("težine") ili dok ne naiđe na stanje gde je neki igrač pobedio. Ovo radi tako što poziva **`get_possible_future_states(old_state: GameState)`** -> **`list[GameState]`** funkciju koja vraća nazad listu svih mogućih budućih stanja, i onda za sve elemente te liste izračunava razliku heuristike računara i igrača.

Razlike heuristike je veća za bolji potez od strane računara, tako da se za najbolji potez vraća stanje sa najvećom vrednošću.

Alfa beta odsecanje smanjuje broj stanja koje algoritam mora da proveri.

Računanje heuristike

Veća vrednost heuristike je loša.

Vrednost heuristike se računa pomoću **`determine_heuristic_for(state: GameState, player: Color)`** -> **`int`** funkcije. Ona:

1. Traži rastojanje od svih ostrva koristeći **`distances_from_islands(state: GameState, player: Color)`** -> **`list[dict[int, list[int]]]`**
2. Računa heuristiku za sva rastojanja
 - a. Ako je povezano sa ostrvom, poziva se **`connected_heuristic(distance: dict[int, list[int]], segment_islands: list[int])`** -> **`int`**
 - b. Ako nije povezano sa ostrvom, poziva se **`unconnected_heuristic(distance: dict[int, list[int]])`** -> **`int`**
3. Vraća najmanju heuristiku u listi

Nepovezana heuristika

Računa se kao zbir proizvoda svih distanca sa brojem ostrva koji se nalaze na toj distanci. Tako bolju heuristiku imaju stanja koja nas približavaju većem broju ostrva

Povezana heuristika

Računa se tako što proveravamo suprotna ostrva za svaki segment. Ako postoji put do njih, dodaje se udaljenost, ali ako ne postoji, onda povećavamo vrednost heuristike.

Prikazivanje table

Na sličan način kao što smo i kreirali tablu, mi inkrementalno pravimo 2D matricu od grafa koja će biti prikazana na ekranu.

Pozivam na funkciju ***print_graph(graph: dict[Point, Node], graph_size: int)*** mi definišemo neke globalne vrednosti koje zavise od veličine grafa (offset, visina grafa, broj kolona...) da bismo ih kasnije referencirali u ostalim funkcijama.

1. ***create_table(graph: dict[Point, Node])*** funkcija kreira praznu 2D matricu i zapisuje simbole iz našeg grafa u adekvatne pozicije
2. ***create_island_table(table: list[list[str]])*** funkcija uzima tabelu koju smo napravili iz *create_table* funkcije i dodaje joj ostrva koje se nalaze sa strane
3. ***create_legend_table(island_table: list[list[str]])*** funkcija dodaje simbole na tabelu sa ostrvima kako bi korisnik lakše mogao da uradi svoj potez

Nakon ovih poziva, *print_graph* funkcija može da odštampa konačnu 2D matricu, tj. tablu, na ekranu.

Provera kraja

Čuvaju se polja koja predstavljaju ostrva za zelenog i crvenog igrača koristeći ***get_green_island_points(size: int) -> list[list[Point]]*** i ***get_red_island_points(size: int) -> list[list[Point]]***, respektivno.

Nakon svakog poteza se proverava da li postoji kontakt između dva (2) susedna ostrva koristeći ***has_winning_bridges(graph: dict[Point, Node], color: Color, island_points: list[list[Point]],) -> bool*** funkciju.

To radimo tako što za svako ostrvo iz naše liste (zelenih ili crvenih) ostrva pravimo listu nesusednih ostrva koristeći ***opposite_sides(island_points: list[list[Point]], side_index: int) -> list[list[Point]]***.

Nakon toga, proveravamo da li je neka tačka tačka it svakog ostrva spojena sa tačkom iz njemu susednih ostrva. To radimo koristeći iterativni DFS algoritam

has_path_same_color(start_node: Node, target_node: Node) -> bool.

Struktura podataka:

Stanje igre

Cela igra može da se predstavi korišćenjem GameState objekta. On sadrži:

- graph - tablu (koja je predstavljena kao graf)
- board_size - veličinu table
- current_player - trenutni igrač, predstavljen kao boja njegovog polja
- main_player - ovo je igrač koji ide prvi, i takođe je uvek ljudsko biće, i takođe je predstavljen kao boja
- opponent - ovo je igrač koji ide drugi, može da bude ljudsko biće ili kompjuter, i takođe je predstavljen kao boja
- turn - indeks poteza

Pozivom **next_turn()** metode mi povećavamo brojač poteza i menjamo boju trenutnog igrača.

Tabla

Za predstavljanje table koristili smo strukturu podataka dictionary (ili hashmap u drugim jezicima) koji za ključ drži tačku (Point), a kao vrednost drži cvor (Node).

Point sadrži dva polja, slovo i broj, i obeležava poziciju kao što je zapisano i na prezentaciji.

Node ili čvor je tipična struktura podataka koje se koristi za implementaciju grafa.

Sadrži dva polja simbol (koji se koristi za grafičko predstavljanje tog čvora) i neighbors (koji predstavlja pokazivače na susedne čvorove).

Simbol je implementiran kao Enum koji sadrži unicode vrednosti emodžija.

Ova struktura podataka nam dozvoljava da:

- a) Lakše pratimo validnost unosa korisnika (unos -> čvor)
- b) Lakše implementiramo algoritme za pretraživanje koje radimo na ovom predmetu unutar čvorova